

Plan Maestro: Agente de Gestión Automatizada de Cartera de ETFs

Basado en criterios cuantitativos, análisis de noticias, y decisiones autónomas

Versión: 1.0

Fecha: 2026-03-19

Autor: Sesión de planificación Gerion + Paduel

Estado: Borrador para revisión y ajuste

Tabla de Contenidos

1. [Visión y Premisas](#1-visión-y-premisas)
2. [Arquitectura General del Sistema](#2-arquitectura-general-del-sistema)
3. [universe de ETFs y universe de activos](#3-universe-de-etfs-y-universe-de-activos)
4. [Criterios Cuantitativos (Modelo de Señales)](#4-criterios-cuantitativos-modelo-de-señales)
 - 4.1 Factores cuantitativos
 - 4.2 Indicadores técnicos
 - 4.3 Puntuación compuesta
5. [Análisis de Noticias y Sentimiento](#5-análisis-de-noticias-y-sentimiento)
 - 5.1 Fuentes de noticias
 - 5.2 Pipeline de NLP
 - 5.3 Señales derivadas de noticias
6. [Gestión de Riesgos](#6-gestión-de-riesgos)
 - 6.1 Métricas de riesgo
 - 6.2 Triggers de intervención
 - 6.3 Rebalanceo táctico
7. [Decisiones y Lógica de Trading](#7-decisiones-y-lógica-de-trading)
 - 7.1 Tipos de decisiones
 - 7.2 Calendario de decisiones
 - 7.3 Órdenes y ejecución
8. [Infraestructura](#8-infraestructura)
 - 8.1 Estado actual del VPS
 - 8.2 Componentes necesarios
 - 8.3 Arquitectura de despliegue
 - 8.4 Costes estimados
9. [Base de Datos](#9-base-de-datos)
10. [Scripts Propuestos (Pseudocódigo y Especificaciones)](#10-scripts-propuestos-pseudocódigo-y-especificaciones)
 - 10.1 Scripts de datos

- 10.2 Scripts de análisis cuantitativo
 - 10.3 Scripts de análisis de noticias
 - 10.4 Scripts de gestión de riesgos
 - 10.5 Scripts de decisiones y trading
 - 10.6 Scripts de reporting
 - 10.7 Scripts de orquestación
11. [OpenClaw como Capa de Orquestación](#11-openclaw-como-capa-de-orquestación)
 12. [Fases de Implementación](#12-fases-de-implementación)
 13. [Limitaciones y Disclaimers](#13-limitaciones-y-disclaimers)
 14. [Próximos Pasos para Revisión](#14-próximos-pasos-para-revisión)

1. Visión y Premisas

1.1 Objetivo del Sistema

Construir un sistema multi-agente basado en OpenClaw que gestione de forma autónoma una cartera de ETFs replicable (ETF replicantes de índices: SPY, QQQ, VEA, VWO, BND, TLT, GLD, etc.) aplicando:

- **Criterios cuantitativos** (momentum, valor, calidad, baja volatilidad, tamaño)
- **Análisis de sentimiento** derivado de noticias financieras
- **Gestión activa de riesgos** con triggers dinámicos
- **Rebalanceo mensual programado** salvo excepciones por riesgo
- **Intervención táctica** cuando el riesgo lo aconseje

1.2 Premisas Fundamentales

1. **El usuario final es Paduel** — inversor minorista en España, timezone Madrid.
2. **No se requiere ejecución automática de órdenes en broker** — el sistema genera recomendaciones y alertas que Paduel ejecuta manualmente, al menos en la fase inicial. Esto elimina complejidad regulatoria y de conexión a brokers.
3. **El VPS actual es el único servidor** disponible — Ubuntu 22.04, 2 vCPU, 3.7 GB RAM, ~29 GB disco libre.
4. **Sin base de datos instalable** — se usará SQLite para persistencia local (compatible con los recursos disponibles).
5. **Sin API key de broker** — las órdenes se comunican por Telegram y se ejecutan manualmente.
6. **El modelo de lenguaje (MiniMax M2.7)** se usa para: análisis de sentimiento, resumen de noticias, generación de informes, y razonamiento sobre decisiones.

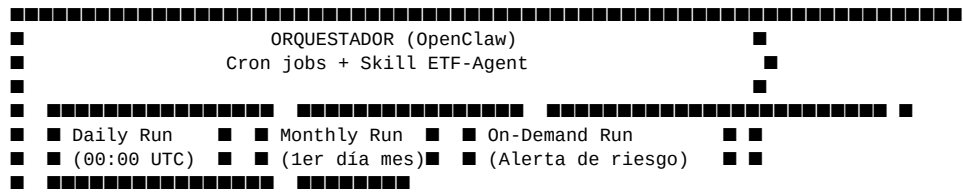
1.3 Alcance vs. No-Alcance

Dentro del alcance	Fuera del alcance (v1)
---	---
Generación de señales cuantitativas	Ejecución automática en broker
Análisis de sentimiento de noticias	Conexión API a broker
Reporting y alertas por Telegram	Trading real sin supervisión
Base de datos local de precios y decisiones	Optimización de impuestos (Modelo 720, D-6)
Gestión de riesgo con triggers	Trading de alta frecuencia
Rebalanceo mensual con excepciones	Acceso a datos de broker (posiciones reales)

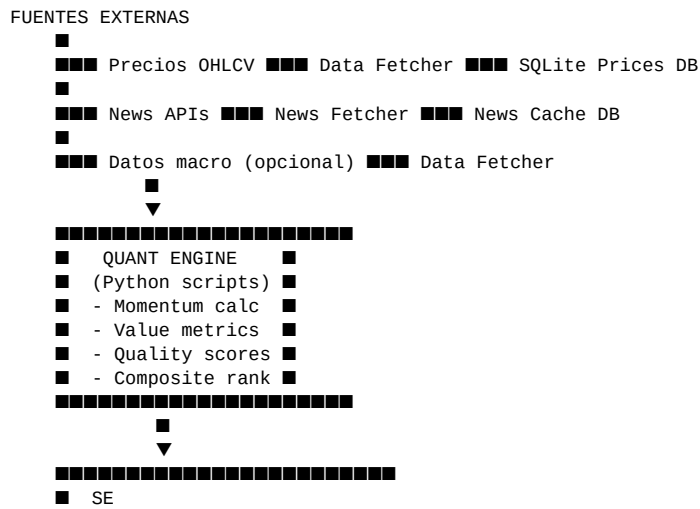
****Nota:**** Si en el futuro se decide conectar a un broker (Interactive Brokers, Alpaca), el sistema está diseñado para poder **■** esa capacidad.

2. Arquitectura General del Sistema

2.1 Diagrama de Componentes



2.2 Flujo de Datos Principal



3. Universe de ETFs y Universe de Activos

3.1 Selección del Universe

El universo inicial se compone de ETFs ampliamente líquidos, replicables por índice, cubriendo los principales factores y clases de activos. Esto permite:

- Diversificación real entre clases de activos (equity, fixed income, commodities)
- Exposición a factores cuantitativos probados
- Costes de transacción bajos (spreads ajustados)
- Dividendos y distribución de rendimientos

3.2 Universe Propuesto (Revisible con Paduel)

ticker	Nombre	Clase	Factor Principal	Liquidez
-----	-----	-----	-----	-----
SPY	SPDR S&P 500 ETF Trust	Equity US	Broad Market	★★★★★
QQQ	Invesco QQQ Trust	Equity US Tech	Momentum/Growth	★★★★★
VEA	Vanguard FTSE Developed Markets	Equity Intl	Valor/Size	★★★★
VWO	Vanguard FTSE Emerging Markets	Equity EM	Valor/Size	★★★★
IVV	iShares Core S&P 500	Equity US	Broad Market	★★★★★
VTI	Vanguard Total Stock Market	Equity US	Broad Market	★★★★★

BND	Vanguard Total Bond Market	Fixed Income	Safety	★★★★★
TLT	iShares 20+ Year Treasury	Fixed Income	Duration/Risk-off	★★★★
LQD	iShares iBoxx Investment Grade	Fixed Income	Credit Quality	★★★★
HYG	iShares iBoxx High Yield	Fixed Income	Carry/Yield	★★★
GLD	SPDR Gold Shares	Commodity	Safe Haven/Inflation	★★★★
SLV	iShares Silver Trust	Commodity	Leverage to Gold	★★★
IAU	iShares Gold Trust	Commodity	Gold (cheaper)	★★★
VNQ	Vanguard Real Estate	Real Estate	Income/Diversification	★★★★
UNH	—	—	ELIMINADO (no es ETF)	—
EFA	iShares MSCI EAFE	Equity Intl	Developed Markets	★★★★
EEM	iShares MSCI Emerging Markets	Equity EM	EM Broad	★★★★
UST	ProShares Ultra 7-10 Year Treasury	Fixed Income	Leverage	★★★
TQQQ	ProShares UltraPro QQQ	Equity US	Leverage 3x	★★
SPXL	Direxion Daily S&P 500 Bull	Equity US	Leverage 3x	★★

3.3 Clasificación por Perfil

Para facilitar la asignación de pesos, se agrupan en perfiles:

- ****REquity Dominant:**** SPY, IVV, VTI (EEUU broad)
- ****Growth/Tech:**** QQQ, TQQQ, SPXL
- ****International Developed:**** VEA, EFA
- ****Emerging Markets:**** VWO, EEM
- ****Fixed Income Safe:**** BND, TLT, LQD
- ****High Yield:**** HYG
- ****Commodities:**** GLD, IAU, SLV
- ****Real Estate:**** VNQ

3.4 Benchmark

El benchmark será una asignación estática 60/40 (60% equity ETFs / 40% fixed income + otros) que sirva como referencia de rebalanceo:

Equity (60%): 30% SPY + 10% QQQ + 10% VEA + 10% VWO
Fixed Income (30%): 15% BND + 10% TLT + 5% LQD
Alternativos (10%): 5% GLD + 5% VNQ

4. Criterios Cuantitativos (Modelo de Señales)

4.1 Factores Cuantitativos

Se usan 5 factores probados en la literatura quant (Fama-French, factores AQR, factores de FTSE Russell):

Qué mide: La tendencia del ETF en los últimos 3, 6 y 12 meses.

Cálculo:

- $\text{MOM}_{3m} = (\text{precio_hoy} / \text{precio_hace_3m}) - 1$
- $\text{MOM}_{6m} = (\text{precio_hoy} / \text{precio_hace_6m}) - 1$
- $\text{MOM}_{12m} = (\text{precio_hoy} / \text{precio_hace_12m}) - 1$
- $\text{Momentum Score} = 0.5 * \text{MOM}_{3m} + 0.3 * \text{MOM}_{6m} + 0.2 * \text{MOM}_{12m}$ (ponderación exponencial)

Interpretación: Mayor score = más fuerza relativa reciente.

Qué mide: Si el ETF está "barato" respecto a fundamentales.

Para equity ETFs: Se usa el P/E del índice subyacente (aproximable via yfinance o ETF proxies).

Para fixed income: Yield spread vs. treasuries del mismo vencimiento.

Para commodities: Precio spot vs. coste de producción (aproximación: ratio precio vs. media móvil de 5 años).

Cálculo: Z-score del P/E o yield respecto al universo. Cuanto más bajo el P/E, mayor score de valor.

Qué mide: Calidad del activo subyacente.

- Para equity: ROE, debt-to-equity del índice (datos trimestrales).
- Para bonds: rating crediticio medio del ETF.
- Para commodities: producción global, reservas.

Fuentes: Alpha Vantage fundamentals, Finnhub, o aproximaciones públicas.

Qué mide: Rendimiento ajustado al riesgo.

Cálculo:

- $\text{Vol_20d} = \text{stddev}(\text{retornos_20d})$
- $\text{Sharpe_60d} = \text{mean}(\text{retornos_60d}) / \text{stddev}(\text{retornos_60d})$ (■■■■■ = 0 para simplificación)
- $\text{LowVol Score} = -\text{Vol_20d} + 0.5 * \text{Sharpe_60d}$

Interpretación: Menor volatilidad = mejor score.

Qué mide: Contribución a la diversificación de la cartera.

- Ajustado por correlación con el resto de la cartera
- ETFs de mercados pequeños/emergentes tienen peso extra por diversificación
- $\text{Size Score} = \text{correlation_benefit} - \text{concentration_penalty}$

Qué mide: Calidad de la tendencia más allá del momentum puro.

Cálculo:

- SMA_50 vs SMA_200 → posición (precio vs medias móviles)
- ADX (Average Directional Index) → fuerza de la tendencia
- $\text{Trend Score} = \text{sign}(\text{SMA_50} - \text{SMA_200}) * \text{ADX} / 100$

4.2 Indicadores Técnicos Complementarios

Estos NO entran en el scoring compuesto pero alimentan las decisiones:

- **RSI(14):** Si >70 sobrecomprado, reduce peso. Si <30 sobrevendido, aumenta peso (hasta umbral).
- **MACD:** Cruces del histograma para timing de entrada/salida táctica.
- **Bollinger Bands:** %B y ancho de bandas para detectar extremos.
- **Drawdown reciente:** Máxima caída desde peak en los últimos 60 días.
- **VIX proxy:** Para equity ETFs, usar el ratio PUT/CALL o derivados como señal de riesgo.

4.3 Puntuación Compuesta

Para cada ETF, se calcula:

$$\text{Raw Score} = \sum (\text{factor_weight}_i \times \text{normalized_factor_score}_i)$$

Donde cada factor se normaliza a Z-scores respecto al universo de ETFs (para que 0 = media del universo, +2 = outlier positivo, -2 = outlier negativo).

El **Composite Score** final se usa para:

1. **Ranking** de ETFs dentro de cada clase

- 2. Ajuste de pesos dentro de la cartera
- 3. Generación de señales de trading

4.4 Tabla de Thresholds de Señales

Composite Score	Señal
-----	-----
> 1.5 (vs benchmark)	**STRONG BUY** — Sobreponderar significativamente
0.5 a 1.5	**BUY** — Sobreponderar
-0.5 a 0.5	**HOLD** — Mantener peso neutro
-0.5 a -1.5	**SELL** — Subponderar
< -1.5	**STRONG SELL** — Eliminar o infraponderar significativamente

5. Análisis de Noticias y Sentimiento

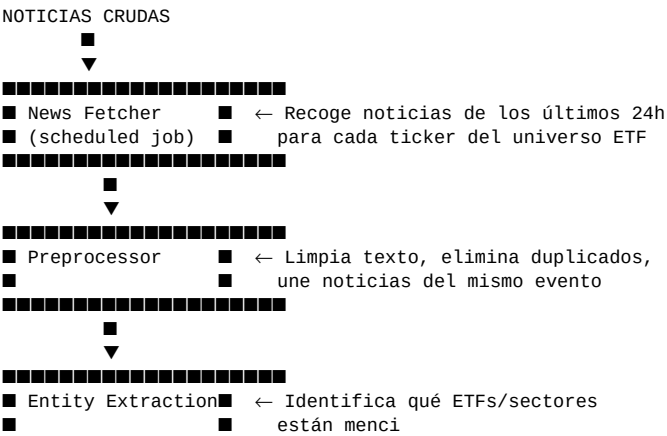
5.1 Fuentes de Noticias

Se usarán múltiples fuentes para redundancia y cobertura:

Fuente	API	Gratis	Cobertura	Limit
-----	-----	-----	-----	-----
Finnhub	https://finnhub.io	Sí (60 req/min)	Global financial news, SEC filings	Buena
Alpha Vantage	https://alphavantage.co	Sí (25 req/day)	News + sentiment scores	Buena
NewsAPI.org	https://newsapi.org	100 req/day	Headlines general	General
GNews	https://gnews.io	100 req/day	News por query	Buena
Marketaux	https://marketaux.com	Plan free	Financial news + sentiment	Beta
EODHD News	https://eodhd.com	20 req/day	News + sentiment	Buena
Yahoo Finance RSS	https://finance.yahoo.com	Sí	News por ticker	Buena (scraping)

Estrategia: Se usa Finnhub como fuente primaria (mejor ratio cobertura/gratis). NewsAPI.org como backup para headlines generales.

5.2 Pipeline de NLP para Sentimiento



5.3 Détalle del Análisis con LLM

Se usará el modelo MiniMax vía la API de OpenClaw. El prompt será estructurado:

Eres un analista financiero especializado en ETFs. Analiza el siguiente artículo

y responde en JSON.

Artículo:
[TEXTO DEL ARTÍCULO]

Responde con este JSON exactamente:

```
{
  "sentiment": "positive|negative|neutral",
  "confidence": 0.0-1.0,
  "impact_direction": "bullish|bearish|neutral",
  "impact_magnitude": "high|medium|low",
  "time_horizon": "short|medium|long",
  "affected_etfs": ["ticker1", "ticker2"],
  "affected_sectors": ["Technology", "Energy", ...],
  "key_takeaway": "resumen en 1-2"
```

5.4 Señales Derivadas de Noticias

Sentiment Agregado (últimos 7d)	Acción
-----	-----
> 0.7 (muy positivo)	Aumentar peso del ETF (+5% adicional)
0.55 - 0.70	Mantener, posible entrada
0.45 - 0.55	Neutral, ignorar para decisión
0.30 - 0.45	Precaución, reducir peso si cuantitativo lo confirma
< 0.30	Reducir peso significativo (-5%) o salir si cuantitativo confirma

5.5 News Sentiment Score por ETF

Se calcula como:

$$\text{News Score ETF}_i = \frac{\sum (\text{sentiment}_j \times \text{confidence}_j \times \text{recency_weight}_j)}{\sum (\text{confidence}_j \times \text{recency_weight}_j)}$$

Donde recency_weight_j es mayor para noticias más recientes (exponencial decay sobre 7 días).

6. Gestión de Riesgos

6.1 Métricas de Riesgo

- **Horizonte:** 1 día y 10 días
- **Nivel de confianza:** 95% y 99%
- **Método:** Paramétrico (media-varianza,  retornos normales) + histórico para validación
- **Cálculo:** $\text{VaR}_{95} = \mu - 1.645 \times \sigma$ (para 1 día)
- Mide la pérdida media cuando se supera el VaR
- Más conservativo que VaR
- $\text{CVaR}_{95} = E[L \mid L > \text{VaR}_{95}]$
- **Max Drawdown:** Máxima caída desde peak histórico en el portfolio
- **Current Drawdown:** Caída actual desde el último peak
- **Recovery Time Estimado:** Ratio entre pérdida y retorno medio mensual
- Para cada ETF, tracked weight vs target weight
- Si drift > umbral (ej: 5% absoluto), señal de rebalanceo obligatoria
- Volatilidad annualized del portfolio completo
- Comparación con benchmark (60/40)
- Si portfolio vol > 1.5x benchmark vol → alerta

6.2 Triggers de Intervención de Riesgo

Estos triggers pueden **invalidar o modificar** las señales cuantitativas normales:

Trigger	Condición	Acción Automática
-----	-----	-----
VaR Breach	VaR 99% 1-día > 3% del portfolio	Reducción automática del 20% de exposición a equity, aumento a BND/GLD
Drawdown crítico	Portfolio -drawdown > 10% desde peak	Reducción agresiva, mover 50% a defensivos
Volatility Spike	VIX proxy > 30 o $\sigma_{\text{portfolio}}$ > 2x histórico 60d	Reducir exposición a equity 25%
Correlación Stress	Correlación entre equity y bonds > 0.7 (típico en crisis)	Reducir ambos, aumentar cash/GLD
Concentration Alert	Cualquier ETF > 40% del portfolio	Rebalanceo obligatorio aunque no sea mes
Liquidity Alert	Para EM/commodity ETFs con spread > 1%	Evitar trading hasta normalización
Sentiment Extreme	News sentiment < 0.2 para >50% del equity exposure	Reducción moderada (10-15%)
Overnight Gap Risk	Mercado Futures caídas > 3% en pre-market	Reducción preventiva antes del open

6.3 Rebalanceo Táctico por Riesgo

Cuando un trigger de riesgo se activa:

```
IF risk_trigger_active:
    CALCULATE risk_adjusted_weights
    COMPARE vs current_weights
    IF abs(diff) > rebalance_cost_threshold:
        GENERATE_RECOMMENDATION(
            action="RISK_REBALANCE",
            reason="<risk_trigger>",
            target_weights=risk_adjusted_weights,
            priority=HIGH,
            execute_before_next_scheduled=True
        )
```

El umbral de coste de transacción para rebalancear se fija en **0.2% del valor del portfolio** — no se recomienda rebalanceo si el coste estimado supera esto.

7. Decisiones y Lógica de Trading

7.1 Tipos de Decisiones

- Se genera cada día automáticamente (cron 00:00 UTC)
- Recoge datos, recalcula scores, actualiza dashboard
- ****No genera órdenes**** — solo señales y tracking
- Si hay trigger de riesgo activo, envía ****alerta inmediata****
- Se ejecuta el ****primer día de cada mes**** (o el siguiente día hábil)
- Revisa weights actuales vs. target weights
- Genera órdenes de rebalanceo si drift > 0.5%
- Combina señales cuantitativas + sentimiento
- Presenta plan de acción a Paduel por Telegram
- Paduel confirma y ejecuta manualmente
- Se activa cuando un trigger de riesgo se dispara
- Envía ****alerta prioritaria**** a Paduel
- Recomendación acci3n correctiva específica
- Si Paduel no responde en 4 horas, se reenvía alerta
- Nunca ejecuta automáticamente (salvo si se configura lo contrario en el futuro)
- Revisi3n completa del sistema

- Backtesting de las señales del último trimestre
- Ajuste de ponderaciones de factores si es necesario
- Revisión del universo ETF

7.2 Calendario de Decisiones

Día/Hora (UTC)	Tipo	Descripción
-----	-----	-----
Daily 00:00	A: Daily Signal	Recálculo de scores, update DB, alert if risk
1er día mes 08:00	B: Monthly Rebalance	Plan de rebalanceo → Telegram a Paduel
Cualquier momento	C: Exception	Alerta riesgo — llega cuando ocurre
Enero, Abril, Julio, Octubre — 1er Lunes 09:00	D: Quarterly Review	Reporte trimestral completo
Semana 52 / Dic 30	—	Reporte anual + planificación año siguiente

7.3 Lógica de Decisión Completa

```
FUNCTION generate_decision(day_type):

    # STEP 1: Load current state
    portfolio = load_portfolio_state()
    prices = load_recent_prices(universe)
    signals = load_quant_signals()
    sentiment = load_sentiment_scores()
    risk_metrics = calculate_risk_metrics(portfolio, prices)

    # STEP 2: Risk check (always first)
    IF risk_metrics.trigger_active:
        risk_action = generate_risk_recommendation(risk_metrics)
        IF day_type == 'daily':
            SEND_ALERT(risk_a
```

7.4 Formato de Recomendación (Output)

[illegible]

8. Infraestructura

8.1 Estado Actual del VPS

```

Hostname:      ubuntu-clawdbot-ger1
OS:            Ubuntu 22.04.5 LTS (Jammy)
Arch:          aarch64 (ARM64)
vCPUs:         2
RAM:           3.7 GB (2.9 GB disponibles)
Swap:          0 (sin swap)
Disco:         38 GB total, 29 GB libres (22%)

```

Python: 3.10.12
 Node.js: v24.13.0 (via nvm)
 pip: 26.0.1
 OpenClaw: Instalado en ~/.nvm/.../openclaw
 Workspace: /home/gerion/.openclaw/workspace
 Credentials: Google OAuth (Gmail, Calendar, Drive)
 Canal: Telegram

8.2 Componentes Necesarios

Componente	Instalación	Propósito
-----	-----	-----
Python 3.10+	Ya instalado ✓	Runtime principal para scripts de análisis
SQLite3	Ya incluido en Ubuntu ✓	Base de datos local
pip3	Ya instalado ✓	Gestor de paquetes Python
yfinance	`pip3 install yfinance`	Datos de precios históricos
pandas	`pip3 install pandas`	Manipulación de datos
numpy	`pip3 install numpy`	Cálculos numéricos
sqlalchemy	`pip3 install sqlalchemy`	ORM para SQLite
schedule	`pip3 install schedule`	Job scheduling simple en Python
requests	`pip3 install requests`	HTTP client para APIs
python-dotenv	`pip3 install python-dotenv`	Variables de entorno
httpx	`pip3 install httpx`	Cliente HTTP async
textblob	`pip3 install textblob`	NLP básico
vaderSentiment	`pip3 install vaderSentiment`	Sentiment redes sociales

Paquete	Instalación	Propósito
-----	-----	-----
transformers (HuggingFace)	`pip3 install transformers torch`	Modelos BERT/FinBERT para sentiment
alpha_vantage	`pip3 install alpha-vantage`	API Alpha Vantage
finnhub-python	`pip3 install finnhub-python`	API Finnhub
backtrader	`pip3 install backtrader`	Backtesting (para quarterly review)
vectorbt	`pip3 install vectorbt`	Backtesting avanzado
matplotlib	`pip3 install matplotlib`	Gráficos para reports
plotly	`pip3 install plotly`	Gráficos interactivos

■■ ****Nota sobre RAM:**** Transformers + PyTorch requieren ~2GB adicional. Con 3.7GB total, se puede instalar pero hay que ejecutarlos con gestión de memoria cuidadosa, o usar la alternativa más ligera de `vaderSentiment` + `textblob` para la Layer 1 de sentiment.

API	Coste	Uso	Sign-up
----	-----	-----	-----
Finnhub	Gratis (60 req/min)	News + sentiment + fundamental	https://finnhub.io
Alpha Vantage	Gratis (25 req/day)	Precios + fundamentals	https://alphavantage.co
NewsAPI.org	Gratis (100 req/day)	Headlines	https://newsapi.org
EODHD	Gratis (20 req/day)	News + sentiment + precios	https://eodhd.com
MiniMax (ya disponible)	Incluido en plan	LLM sentiment + reports	Ya configurado

8.3 Arquitectura de Despliegue

```
/home/gerion/etf-agent/
■■■ config/
■   ■■■ .env                # API keys y configuración sensible
■   ■■■ universe.json       # Lista de ETFs del universo
■   ■■■ factors.json        # Pesos de factores cuantitativos
■   ■■■ risk_thresholds.json # Umbrales de riesgo
■   ■■■ cron_schedule.json  # Definición de jobs programados
■■■ data/
■   ■■■ prices.db           # SQLite: OHLCV diarios
■   ■■■ news.db             # SQLite: noticias cacheadas
■   ■■■ signals.db
```

8.4 Estructura de Base de Datos (SQLite)

```
-- Tabla: prices
-- OHLCV diario por ETF
CREATE TABLE prices (
    ticker    TEXT NOT NULL,
    date      TEXT NOT NULL, -- YYYY-MM-DD
    open      REAL,
    high      REAL,
    low       REAL,
    close     REAL,
    adj_close REAL,
    volume    INTEGER,
    PRIMARY KEY (ticker, date)
);

-- Tabla: news_cache
-- Noticias cacheadas con sentiment calculado
CREATE TABLE news_cache (
    id        TEXT PRIMARY KEY, -- hash del artículo
    source    TEXT,
    published TEXT
```

9. Base de Datos (Ampliación)

9.1 Gestión de Datos

La base de datos SQLite (data/prices.db, data/news.db, etc.) se gestiona con SQLAlchemy como ORM. Cada tabla tiene funciones helper dedicadas en src/db_manager.py:

```
# Ejemplo de funciones del db_manager
def save_price(ticker: str, date: str, ohlcv: dict): ...
def get_prices(ticker: str, start_date: str, end_date: str) -> pd.DataFrame: ...
def get_latest_price(ticker: str) -> dict: ...
def save_signal(ticker: str, date: str, signals: dict): ...
def save_news(news_item: dict) -> bool: ... # True si es nuevo, False si duplicado
def get_news_for_ticker(ticker: str, days: int = 7) -> list: ...
def get_portfolio_decision(date: str, decision_type: str) -> dict: .
```

9.2 Actualización de Datos

Datos	Frecuencia	Fuente	Retention
-----	-----	-----	-----
OHLCV diario	Diario (post-mercado, 23:00 UTC)	yfinance (+ Alpha Vantage backup)	Indefinido
News	Cada 4h durante trading hours	Finnhub + NewsAPI	90 días
Signals calculados	Diario (post-actualización precios)	Engine interno	Indefinido
Sentiment agregado	Diario	Engine interno	365 días
Decisiones	Por evento	Engine interno	Indefinido
Portfolio holdings	Manual (input Paduel)	Input del usuario	Indefinido

9.3 Datos Macroeconómicos (Futuro)

En fases posteriores se puede añadir:

- VIX (^VIX) — proxy de miedo de mercado
- DXY (US Dollar Index) — para impacto en EM
- Treasury yields (^TNX, ^TYX) — para análisis de duration
- Indicadores de crédito (HYG, LQD spreads)
- PMI global (OECD data — gratuito)

10. Scripts Propuestos (Pseudocódigo y Especificaciones)

10.1 Scripts de Datos

Propósito: Descargar precios diarios de cierre ajustado para todos los ETFs del universo.

Trigger: Diario a las 23:00 UTC (después del cierre de NYSE).

Pseudocódigo:

SCRIPT: fetch_prices.py
INPUT: universe.json (lista de tickers)
OUTPUT: prices.db (tabla prices)

1. CARGAR lista de tickers desde universe.json
2. PARA CADA ticker EN universe:
 - a. Intentar con yfinance:
 - descargar con start = (hoy - 400 días), end = hoy
 - guardar OHLCV en prices.db (INSERT OR REPLACE)
 - b. SI yfinance falla:
 - intentar Alpha Vantage (con API key de .env)
 - mapear respuesta a formato estándar
 - guardar en prices.db
 - c. SI ambas fallan:

API Keys necesarias: Alpha Vantage (gratis, 25 req/day — suficiente para uso diario si se limita a los 20 ETFs).

Propósito: Recoger noticias financieras de los últimos 3 días para los tickers del universo.

Trigger: Cada 6 horas (00:00, 06:00, 12:00, 18:00 UTC).

Pseudocódigo:

SCRIPT: fetch_news.py
INPUT: universe.json
OUTPUT: news.db (tabla news_cache)

1. CARGAR Finnhub API key desde .env
2. PARA CADA ticker EN universe:
 - a. LLAMAR Finnhub /company-news:
 - from = (hoy - 3 días)
 - to = hoy
 - b. PARA CADA artículo recibido:
 - generar hash = SHA256(url + title)
 - SI hash NO existe en news_cache:
 - guardar en news_cache con fields:
id, source, published, title, summary, tickers=[ticker],
sentiment_score=NULL, s

Propósito: Permitir a Paduel actualizar manualmente sus posiciones reales en la cartera.

Trigger: Manual, via comando OpenClaw.

SCRIPT: update_portfolio_manual.py
INPUT: Comando del usuario con posiciones
OUTPUT: portfolio.db (tabla portfolio_holdings)

1. RECIBIR input del usuario:

Formato esperado: "SPY:100,BND:200,GLD:50"
2. VALIDAR:
 - tickers existen en universe
 - shares > 0
 - avg_cost es numérico positivo

3. CALCULAR valores:
 - Obtener último precio de prices.db
 - Calcular valor_actual = shares × price
 - Calcular P&L = valor_actual - (shares × avg_cost)
4. GUARDAR en portfolio_holdin

10.2 Scripts de Análisis Cuantitativo

Propósito: Calcular scores de momentum para cada ETF.

SCRIPT: calculate_momentum.py
 INPUT: prices.db (precios de últimos 400 días)
 OUTPUT: signals.db (columna mom_score)

PARÁMETROS CONFIGURABLES (en factors.json):

- MOM_3M_WEIGHT = 0.5
- MOM_6M_WEIGHT = 0.3
- MOM_12M_WEIGHT = 0.2
- MOM_NORMALIZATION_WINDOW = 60 (días para z-score)

1. PARA CADA ticker EN universe:
 - a. CARGAR serie de precios de cierre (últimos 400 días)

- b. CALCULAR retornos:
 - ret_3m = (precio_hoy / precio_60d_antes) - 1
 - ret_6m = (precio_hoy / precio

SCRIPT: calculate_factors.py
 INPUT: prices.db, datos fundamentales (Alpha Vantage)
 OUTPUT: signals.db (todas las columnas de factores)

SUB-SCRIPTS (llamados en paralelo o secuencial):

- A) value_factor():
 - Para equity ETFs: P/E ratio del índice subyacente
 - Fuente: Alpha Vantage /stock_news_sentiment (para sentiment) o aproximación via yfinance (ticker.info['trailingPE'])
 - Normalización: z-score del P/E inverso (P/E bajo = score alto)

- B) quality_factor():
 - ROE, Debt-to-Equi

SCRIPT: generate_composite_signals.py
 INPUT: signals.db (todos los factores)
 OUTPUT: signals.db (composite score + signal_label)

CONFIG (factors.json):

```
FACTOR_WEIGHTS = {
    'momentum': 0.25,
    'value': 0.20,
    'quality': 0.20,
    'lowvol': 0.15,
    'trend': 0.10,
    'size': 0.10
}
```

```
SIGNAL_THRESHOLDS = {
    'STRONG_BUY': 1.5,
    'BUY': 0.5,
    'HOLD': -0.5,
    'SELL': -1.5
}
```

1. PARA CADA ticker:
 - a. CARGAR scores de cada factor (z-scores)
 - b. CALCULAR composite:

10.3 Scripts de Análisis de Noticias

SCRIPT: analyze_sentiment_ml.py
 INPUT: news.db (artículos sin sentiment_score)
 OUTPUT: news.db (actualizado con sentiment_ml_score)

LIBRERÍAS: vaderSentiment (VADER) para texto financiero/twitter,
 o cardiffnlp/twitter-roberta-base-sentiment para mayor precisión

1. CARGAR artículos de news_cache donde sentiment_score IS NULL
2. PARA CADA artículo:

```

a. COMBINAR title + summary (max 512 tokens)

b. APLICAR VADER o modelo HF:
  - scores = sentiment_analyzer.polarity_scores(
SCRIPT: analyze_sentiment_llm.py
INPUT: news.db (artículos con sentiment_ml_score)
OUTPUT: news.db (actualizado con llm_analysis JSON)

MODELO: MiniMax M2.7 (via OpenClaw/OpenAI-compatible API)

RATE LIMIT: Max 50 artículos por ejecución (para no agotar la ventana 5h)

1. CARGAR artículos de news_cache:
  - Donde llm_analysis IS NULL
  - Ordenados por published DESC
  - Limite: 50

2. CONSTRUIR prompt por lotes (batch de 5-10 artículos por llamada):
Eres analista financiero. Para cada artículo, responde en JSON.

1. [title1] - [summary1]
2. [title2] - [summary2]

Responde con un array JSON:

{"index": 0, "sentiment": "...", "confidence": 0.X,
"impact_direction": "...", "impact_magnitude": "...",
"affected_etfs": [...], "key_takeaway": "..."},

3. LLAMAR al LLM (usando httpx hacia la API de MiniMax)

4. PARSEAR respuesta JSON

5. PARA CADA resultado:
  - Actualizar llm_analysis = JSON.stringify(resultado)
  - Si hay affected_etfs mencionados, guardar relación en sentiment_scores

6. LOG: articles_processed, llm_calls, errors
SCRIPT: aggregate_sentiment.py
INPUT: news.db (sentiment_scores por artículo)
OUTPUT: sentiment_scores.db (agregados por ETF y ventana)

VENTANAS: 7 días y 30 días

1. PARA CADA ticker EN universe:
  PARA CADA ventana (7d, 30d):
a. CARGAR articulos de news_cache para el ticker
  dentro de la ventana temporal

b. SI hay artículos:
  - avg_sentiment = mean([s.sentiment_score for s in articles])
  × weights (recency decay: exp(-days_ago/3))
  - positive_pct = % de artícu

```

10.4 Scripts de Gestión de Riesgos

```

SCRIPT: calculate_var_cvar.py
INPUT: prices.db, portfolio_holdings (pesos actuales)
OUTPUT: risk_metrics (JSON)

METODO: Simulación histórica (más robusto que paramétrico)

1. CARGAR precios diarios últimos 252 días (1 año trading)

2. CARGAR pesos actuales de la cartera

3. CALCULAR retornos diarios de cada ETF

4. SIMULAR portfolio returns:
  portfolio_returns = Σ (weight_i × return_i)

5. CALCULAR VaR:
  - Ordenar returns de peor a mejor
  - VaR_95_1d = percentile(returns, 5) - pérdida en
SCRIPT: calculate_drawdown.py
INPUT: prices.db, portfolio_holdings

```

OUTPUT: drawdown_metrics (JSON)

1. CARGAR serie de precios de portfolio (valor total = Σ weights $\times$ prices)
2. CALCULAR running peak:
 - peak[t] = max(precio[0:t])
3. CALCULAR drawdown en cada día:
 - dd[t] = (precio[t] - peak[t]) / peak[t]
4. EXTRAER métricas:
 - max_drawdown = min(dd) (peor caída desde peak)
 - current_drawdown = dd[hoy]
 - days_in_drawdown = días desde que empezó el drawdown actual
 - recove

SCRIPT: evaluate_risk_triggers.py

INPUT: risk_metrics.json, sentiment_scores, market_data

OUTPUT: risk_events.db, alertas

CONFIG (risk_thresholds.json):

```
TRIGGERS = {
  "var_breach": {
    "metric": "var_99_1d",
    "threshold": -0.03, # -3%
    "action": "REDUCE_RISK"
  },
  "drawdown_critical": {
    "metric": "max_drawdown",
    "threshold": -0.10,
    "action": "REDUCE_AGGRESSIVE"
  },
  "vol_spike": {
    "metric": "vol_annual",
    "threshold": 0.15, # > 15% v
```

SCRIPT: calculate_risk_adjusted_weights.py

INPUT: signals.db (composite scores), risk_metrics, portfolio_holdings

OUTPUT: target_weights_risk_adjusted.json

LÓGICA:

1. EMPEZAR con weights del composite score ranking
2. APLICAR ajustes de riesgo:
 - a. Si VaR alto: reducir pesos de ETFs de alta volatilidad (QQQ, TQQQ)
 - b. Si drawdown > 5%: aumentar peso de defensivos (BND, GLD)
 - c. Si concentration > 40%: reducir █████ ETF
 - d. Si sentiment negativo: reducir ETF afectado proporcionalmente
3. N

10.5 Scripts de Decisiones y Trading

SCRIPT: generate_daily_signal.py

INPUT: (todo calculado previamente)

OUTPUT: Telegram report diario + signals.db update

TRIGGER: Cron 00:05 UTC daily

1. CARGAR state:
 - latest_signals = signals.db (más recientes)
 - sentiment_scores = sentiment_scores.db (ventana 7d)
 - risk_metrics = calculate_var_cvar()
 - portfolio_state = portfolio.db (último)

2. EVALUAR triggers de riesgo:

- evaluate_risk_triggers()
- SI trigger activo:
 - ENVIAR alerta prioritaria por Telegram

SCRIPT: generate_monthly_rebalance.py

INPUT: (todo calculado + approval workflow)

OUTPUT: Telegram rebalanceo mensual

TRIGGER: Cron 08:00 UTC el día 1 del mes

1. EJECUTAR pipeline completo:
 - fetch_prices.py (si no se ejecutó en últimas 12h)
 - calculate_all_factors.py
 - generate_composite_signals.py
 - aggregate_sentiment.py
 - calculate_var_cvar.py + drawdown.py
 - calculate_risk_adjusted_weights.py
2. CALCULAR target weights:

a. Base: composite scores → ranking de ETFs
SCRIPT: execute_approval.py
INPUT: command from Telegram (via OpenClaw skill handler)
OUTPUT: Actualización de portfolio_state

HANDLER para comandos:
/confirm_rebalance [monthly_id]
/modify_weights [monthly_id] [ticker:weight,...]
/skip_rebalance [monthly_id] [reason]

1. PARSEAR comando y monthly_id
2. SI confirm:
 - a. Marcar approved=1 en portfolio_state
 - b. Generar instrucciones de trading detalladas:
 - Para cada trade: TICKER, ACCIÓN (BUY/SELL), %, €
 - Precio estimado

10.6 Scripts de Reporting

SCRIPT: generate_quarterly_review.py
INPUT: 3 meses de datos
OUTPUT: Reporte trimestral completo

TRIGGER: Primer lunes de Enero, Abril, Julio, Octubre a las 09:00 UTC

CONTENIDO:

1. PERFORMANCE SUMMARY:
 - Rendimiento del portfolio vs benchmark (60/40)
 - Attribution: qué ETFs contribuyeron más/menos
 - Alpha generado: portfolio_return - benchmark_return
2. SIGNAL ACCURACY:
 - Backtesting simple: qué señales BUY se movieron positivamente
 - Qué señales SELL evitaron pérdidas
 - Acc

SCRIPT: generate_annual_report.py
INPUT: 12 meses de datos
OUTPUT: Reporte anual

CONTENIDO:

- Performance año completo
- Gráfico de equity curve (portfolio vs benchmark)
- Estadísticas de risk-adjusted returns
- Decisiones de rebalanceo ejecutadas
- Lecciones aprendidas
- Plan para siguiente año

10.7 Scripts de Orquestación

SCRIPT: run_daily_pipeline.py
TRIGGER: Cron 00:05 UTC daily

ORDEN DE EJECUCIÓN:

1. fetch_prices.py # 2-5 min
2. calculate_momentum.py # 1 min
3. calculate_factors.py # 3-5 min
4. generate_composite_signals.py # 30 seg
5. analyze_sentiment_ml.py # 2-3 min (batch rápido)
6. aggregate_sentiment.py # 1 min
7. calculate_var_cvar.py # 1 min
8. calculate_drawdown.py # 30 seg
9. evaluate_risk_triggers.py # 30 seg
10. generate_daily_signal.py # 1 min (Telegram)
11. log_execution_stats() # 10 seg

SCRIPT: run_monthly_pipeline.py
TRIGGER: Día 1 del mes, 08:00 UTC

ORDEN DE EJECUCIÓN:

1. fetch_prices.py
2. [todos los de daily, para tener datos frescos]
3. calculate_risk_adjusted_weights.py
4. generate_monthly_rebalance.py # Genera plan y envía Telegram
5. WAIT for user approval (48h timeout)
6. execute_approval.py # Cuando llega /confirm

SKILL: etf-agent

TRIGGERS: "análisis ETF", "cartera", "rebalanceo", "/etf"

COMMANDS:


```

/etf status      → Ver estado actual de cartera y señales
/etf signals     → Ver señales de todos los ETFs
/etf risk        → Ver métricas de riesgo actuales
/etf sentiment   → Ver sentiment agregado por ETF
/etf update X:Y,Z:W → Actualizar posiciones manuales
/etf rebalance   → Forzar pipeline mensual
/etf report      → Generar report ad-hoc

```

HANDLER:

1. Parsear comando y argum

11. OpenClaw como Capa de Orquestación

11.1 Diseño del Skill ETF-Agent

Se creará un skill OpenClaw (skills/etf-agent/SKILL.md) que sirva como interfaz principal:

```

skills/etf-agent/
■■■■ SKILL.md (interfaz con OpenClaw)
■■■■ scripts/
■ ■■■■ etf_status.sh      # Consulta estado rápido
■ ■■■■ etf_signals.sh    # Lista de señales
■ ■■■■ etf_risk.sh       # Métricas de riesgo
■ ■■■■ etf_report.sh     # Genera report ad-hoc
■■■■ references/
■■■■ universe.json       # ETF universe
■■■■ thresholds.json     # Thresholds de decisión
■■■■ commands.md         # Referencia de comandos

```

11.2 Integración con Cron

En el archivo de cron de OpenClaw (~/.openclaw/cron/), se registrarán los jobs:

```

[
{
  "id": "etf-daily-signal",
  "schedule": "0 0 * * *",
  "task": "run_daily_pipeline",
  "enabled": true,
  "notify": "on_failure"
},
{
  "id": "etf-news-fetch",
  "schedule": "0 0,6,12,18 * * *",
  "task": "fetch_news",
  "enabled": true,
  "notify": "on_failure"
},
{
  "id": "etf-monthly-rebalance",
  "schedule": "0 8 1 * *",
  "task": "run_monthly_pipeline",
  "enabled": true,
  "notify": "always"
},
{
  "id": "etf-quarterly-review",
  "sche

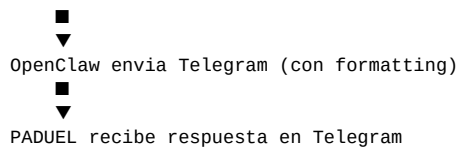
```

11.3 Flujo de Decisión con OpenClaw

```

PADUEL → "Cómo está mi cartera ETF?"
■
▼
OpenClaw recibe mensaje
■
▼
Skill etf-agent detecta comando (o intent)
■
▼
run etf_status.sh → consulta prices.db + signals.db
■
▼
Formatea respuesta con:
- Top signals (top BUY / SELL)
- Risk status (VaR, drawdown)
- Drift vs target weights

```



12. Fases de Implementación

Fase 0: Setup y Fundamentos (Semanas 1-2)

Objetivo: Tener la infraestructura básica funcionando.

Tareas:

- [] Crear directorio `/home/gerion/etf-agent/` con estructura
- [] Instalar dependencias Python (`pip install -r requirements.txt`)
- [] Crear `config/universe.json` con universo de ETFs
- [] Configurar API keys (Finnhub, Alpha Vantage, NewsAPI — todas gratuitas)
- [] Crear schemas de SQLite (ejecutar `init\_db.sh`)
- [] Configurar `.env` con API keys
- [] Hacer `test\_api\_keys.sh` para verificar conectividad
- [] Crear skill `etf-agent` en OpenClaw workspace

Entregable: Sistema que puede descargar precios y mostrar signals básicas.

Fase 1: Pipeline Cuantitativo Completo (Semanas 3-4)

Objetivo: Scoring cuantitativo operativo.

Tareas:

- [] Implementar `calculate\_momentum.py` completo
- [] Implementar `calculate\_factors.py` (value, quality, lowvol, trend)
- [] Implementar `generate\_composite\_signals.py`
- [] Crear `generate\_daily\_signal.py` básico
- [] Integrar con OpenClaw cron para ejecución diaria
- [] Primer test real con datos de precios

Entregable: Daily signal por Telegram con ranking de ETFs.

Fase 2: Gestión de Riesgos (Semanas 5-6)

Objetivo: Módulo de riesgo operativo.

Tareas:

- [] Implementar `calculate\_var\_cvar.py`
- [] Implementar `calculate\_drawdown.py`
- [] Crear `evaluate\_risk\_triggers.py`
- [] Sistema de alertas por Telegram cuando hay trigger
- [] `calculate\_risk\_adjusted\_weights.py`
- [] Integrar con daily pipeline

Entregable: Risk dashboard y alertas automáticas.

Fase 3: Análisis de Noticias y Sentiment (Semanas 7-8)

Objetivo: Sentiment engine operativo.

Tareas:

- [] Implementar `fetch\_news.py` (Finnhub + NewsAPI fallback)
- [] Implementar `analyze\_sentiment\_ml.py` (VADER)
- [] Implementar `analyze\_sentiment\_llm.py` (MiniMax API)
- [] Crear `aggregate\_sentiment.py`
- [] Integrar sentiment scores con decision engine
- [] Test de sentimiento con noticias reales

Entregable: Sentiment scores por ETF en daily report.

Fase 4: Rebalanceo Mensual (Semanas 9-10)

Objetivo: Workflow completo de rebalanceo.

Tareas:

- [] Implementar `generate\_monthly\_rebalance.py`
- [] Crear `execute\_approval.py` (handlers /confirm, /modify, /skip)
- [] Sistema de espera de confirmación de Paduel
- [] Integrar con skill etf-agent para comandos manuales
- [] Test del workflow completo con datos simulados

Entregable: Paduel puede confirmar un rebalanceo desde Telegram.

Fase 5: Input Manual de Cartera + Reporting (Semanas 11-12)

Objetivo: Tracking real de la cartera.

Tareas:

- [] Crear `update\_portfolio\_manual.py`
- [] Implementar quarterly review (`generate\_quarterly\_review.py`)
- [] Crear annual report
- [] Dashboard visual (gráficos via Telegram)
- [] Backtesting básico de señales (ratio de accuracy)

Entregable: Sistema completo con reporting y tracking real.

Fase 6: Optimización y polish (Mes 4+)

Objetivo: Afinar y mejorar.

Tareas:

- [] Backtesting con `backtrader` o `vectorbt`
- [] Ajuste de ponderaciones de factores basado en resultados
- [] Añadir más fuentes de datos si es necesario
- [] Considerar conexión a broker API (Interactive Brokers o Alpaca)
- [] Considerar base de datos más robusta (PostgreSQL) si SQLite es bottleneck

Entregable: Sistema maduro con proceso de mejora continua

12. Fases de Implementación

13. Limitaciones y Disclaimers

13.1 Limitaciones Técnicas

1. **RAM limitada (3.7 GB):** No es posible ejecutar modelos LLM grandes localmente (GPT-J, LLaMA). Todo el análisis LLM se delega a la API de MiniMax.
2. **Sin GPU:** No se pueden usar modelos transformers pesados para sentiment (BERT, FinBERT). Se usa VADER + MiniMax como alternativa.
3. **SQLite:** Funciona bien para datasets pequeños-medianos. Con años de datos y miles de artículos, puede haber degradación. Migración a PostgreSQL es posible si el sistema escala.
4. **Noexec en /tmp:** En algunos VPS linux, /tmp se monta como noexec. Los scripts deben usar directorios escribibles por el usuario.
5. **APIs gratuitas tienen rate limits:** Alpha Vantage (25/day) es suficiente para uso diario si se cachea bien. NewsAPI (100/day) es generoso. Finnhub (60/min) es excelente.

13.2 Limitaciones de Datos

1. **Datos de cierre vs. tiempo real:** Los precios son de cierre (after market). No hay datos intradía sin APIs de pago.
2. **Sin datos de fundamentales en tiempo real:** Los datos de P/E, ROE, etc. son trimestrales y pueden estar desactualizados.
3. **Sentiment LLM depende de calidad de noticias:** Si las APIs no traen noticias suficientes, el sentiment score es ruidoso.

13.3 Limitaciones de Trading

1. **Sin ejecución automática:** El sistema genera recomendaciones. Paduel ejecuta manualmente. Esto significa:
 - No hay latency entre señal y ejecución
 - El factor humano puede retrasar o no ejecutar
 - No hay deslizamiento (slippage) controlado
2. **Sin acceso a posiciones reales del broker:** El portfolio se mantiene manualmente. Si Paduel olvida actualizar posiciones, el sistema trabaja con datos desactualizados.
3. **Costes de transacción reales no modelados:** Se usan aproximaciones (0.1% por trade). Los costes reales varían por broker y volumen.

13.4 Disclaimers Legales

****■ AVISO IMPORTANTE:****

Este sistema es una herramienta de apoyo a la decisión de inversión. No constituye asesoramiento financiero profesional.

Los resultados pasados no garantizan rendimientos futuros.

El trading de instrumentos financieros conlleva riesgos, incluyendo la pérdida parcial o total del capital invertido.

Siempre consulta con un asesor financiero cualificado antes de tomar decisiones de inversión.

Este sistema no garantiza rentabilidad ni protege contra pérdidas.

14. Próximos Pasos para Revisión

14.1 Decisiones Pendientes con Paduel

Antes de empezar a implementar, necesitamos definir juntos:

1. **Capital a gestionar:** ¿Cantidad orientativa de la cartera? (para calibrar tamaños de posición y relevancia de costes de transacción)
2. **Broker:** ¿Desde qué broker opera Paduel?

- Si es ****DEGIRO****: No hay API. Todo será manual + recomendaciones via Telegram.
- Si es ****Interactive Brokers****: Podemos explorar conexión API (requiere cuenta IBKR).
- Si es ****MyInvestor / otros****: Averiguamos si tienen API.
- ****Decisión:**** ¿Conectar API de broker o seguir con ejecución manual?

3. **universo ETF inicial:** ¿Los 20 ETFs listados en sección 3, o quiere reducir/ampliar?

- ¿Incluir ETFs apalancados (TQQQ, SPXL)? Son más arriesgados.
- ¿ETF de ■■■■■■■■■■■■■■■■■■■■■■ (no)?
- ¿ETFs sectoriales específicos (tecnología, healthcare)?

4. **Horizonte temporal:** ¿Inversión a qué plazo?

- < 2 años → Menos peso en equity, más en fixed income
- 5-10 años → Equity heavy, ignorar volatility a corto
- > 10 años → Máximo equity, ignorar drawdowns

5. **Tolerancia al riesgo:** Cuestionario breve:

- ¿Cuánto perderías sin panic sell si la cartera cae -20%?
- ¿Prefieres 8% anual con vol 12% o 5% anual con vol 5%?
- ¿Has vivido un crash de bolsa (2008, 2020)? ¿Qué hiciste?

6. **Perfil de intervención:**

- ¿Quieres recibir TODAS las alertas de riesgo o solo las críticas?
- ¿Prefieres más señales aunque sean ruido, o menos señales más seguras?
- ¿Quieres poder■■■ una señal del sistema si no estás de acuerdo?

7. **Benchmarks:** ¿Qué índice quieres superar como referencia?

- 60/40 (acciones Bonos)
- S&P; 500 solo
- MSCI World
- ¿Otro?

8. **Budget para APIs de pago:** ¿Estás dispuesto a gastar en APIs si las gratuitas se quedan cortas?

- Ej: Polygon.io (~\$50/mes) para datos de calidad
- Ej: Alpaca (free tier disponible) para trading automático

14.2 Plan de Revisión Sugerido

Sesión 1 (30-45 min): Revisar este documento + responder preguntas de la sección 14.1

Sesión 2 (30 min): Validar universo ETF + perfiles de riesgo + broker

Sesión 3 (20 min): Definir umbrales finales (thresholds de señales, risk triggers)

Tras Sesión 3: Inicio de implementación Fase 0

14.3 Checklist de Arranque

ANTES DE EMPEZAR FASE 0:

- Decisión sobre broker (manual vs API)
- Confirmación de universo ETF (lista ajustada)
- API keys solicitadas: Finnhub + Alpha Vantage + NewsAPI
- Tolerancia al riesgo definida (3 preguntas)
- Benchmark confirmado
- Presupuesto APIs de pago (sí/no y cuánto)
- Espacio en disco confirmado para ETL agent (~1GB estimado para 1 año de datos)

Anexo A: Referencias Rápidas

APIs gratuitas recomendadas

API	Link	Rate Limit	Mejor para
-----	-----	-----	-----
Finnhub	https://finnhub.io	60 req/min	News, sentiment
Alpha Vantage	https://alphavantage.co	25 req/day	Precios, fundamentales
NewsAPI	https://newsapi.org	100 req/day	Headlines
yfinance	PyPI	Sin límite*	Precios históricos
EODHD	https://eodhd.com	20 req/day	Precios + news backup

*yfinance usa scraping de Yahoo Finance — puede ser inestable

Librerías Python recomendadas

```
yfinance>=0.2.36
pandas>=2.0
numpy>=1.24
sqlalchemy>=2.0
requests>=2.31
httpx>=0.25
schedule>=1.2
python-dotenv>=1.0
vaderSentiment>=3.3
textblob>=0.17
pandas-ta>=0.8
matplotlib>=3.7
```

Comandos de instalación rápida

```
mkdir -p /home/gerion/etf-agent
cd /home/gerion/etf-agent
mkdir -p config data src logs tests scripts skills

pip3 install yfinance pandas numpy sqlalchemy requests httpx \
    schedule python-dotenv vaderSentiment textblob pandas-ta \
    matplotlib python-dotenv

# Test yfinance
python3 -c "import yfinance as yf; print(yf.Ticker('SPY').history(period='5d'))"
```

Documento generado el 2026-03-19. Sujeto a revisión tras discusión con Paduel.